Software Project Management

Fundamentals

Formal Project Management

Advantages of Using Formal Project Management

- Better control of financial, physical, and human resources
- Improved customer relations
- Shorter development times
- Lower costs
- Higher quality and increased reliability
- Higher profit margins
- Improved productivity
- Better internal coordination
- Higher worker morale (less stress)
  - Less “death marches”
  - Less overworked personnel

What Helps Projects Succeed?*

1. Executive support
2. User involvement
3. Experienced project manager
4. Clear business objectives
5. Minimized scope
6. Standard software infrastructure
7. Firm basic requirements
8. Formal methodology
9. Reliable estimates
10. Other criteria, such as small milestones, proper planning, competent staff, and ownership

*The Standish Group, “Extreme CHAOS,” (2001).

Conventional Software Management Performance

Barry Boehm's “Industrial Software Metrics Top 10 List”:

1. Finding and fixing a software problem after delivery costs 100 times more than finding and fixing the problem in early design phases
2. You can compress software development schedules 25%, but no more
3. For every $1 you spend on development, you will spend $2 on maintenance
4. Software development and maintenance costs are primarily a function of source lines of code.
5. Variations among people account for the biggest difference in software productivity; hire good people to succeed.

Conventional Software Management Performance

Barry Boehm's “Industrial Software Metrics Top 10 List”:

6. The overall ratio of software to hardware costs is still growing.
7. Only about 15% of software development effort is devoted to programming
8. Software systems and products typically cost 3 times as much per SLOC as individual software programs. Software system products (system of systems) costs 9 times as much
9. Walkthroughs catch 60% of the errors
10. 80% of the contributions comes from 20% of the contributors.

First Principles

- One size does not fit all
- Spectrums
  - Project types
  - Sizes
  - Formality and accuracy

Strategy

Hope is not a strategy.

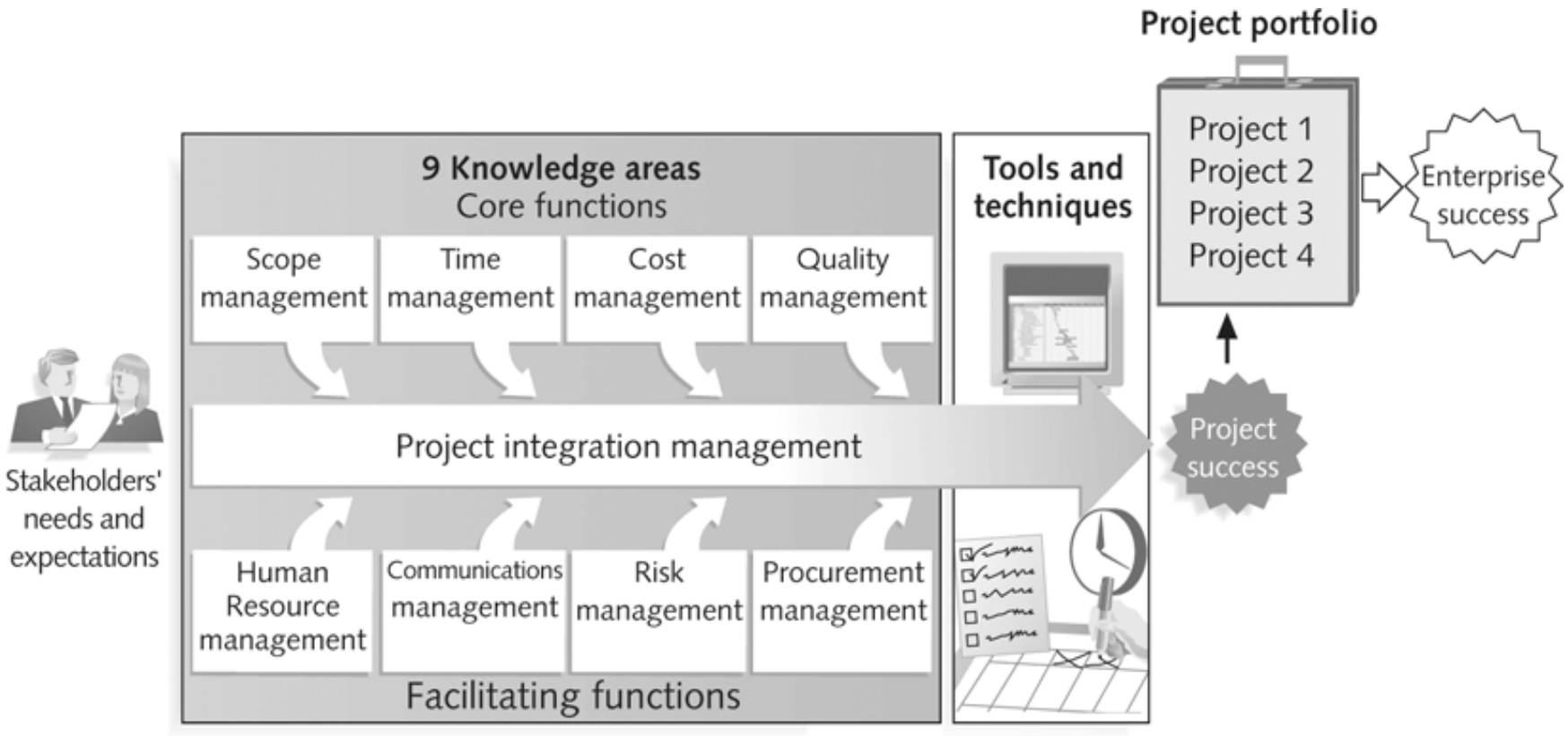
So what is our strategy?

- Classic Mistake Avoidance
- Development Fundamentals
- Risk Management
- Schedule-Oriented Practices

PMI's 9 Knowledge Areas

- Project ***integration*** management
- Scope
- Time
- Cost
- Quality
- Human resource
- Communications
- Risk
- Procurement

Project Management Framework



What is a project life cycle?

- The project life cycle is a collection of sequential or overlapping project phases
  - The phases divide the project into logical blocks of related activities
  - This division into phases simplifies management, planning, and control
- Phases within the project are defined by technical information transfer or technical component hand-off
  - Example: Inception and elaboration phases in the Unified Process
  - Example: Releases in Agile life cycles

Phases

- The completion and approval of one or more *deliverables* (defined as measurable, verifiable work products) defines the endpoint of a project phase
- Different phases can have different relationships among themselves, even within the same project
  - *Sequential relationship*. A phase starts only when the previous phase is complete
  - *Overlapping relationship*. A new phase can be planned and started before the previous phase is complete

PMBOK project life cycles

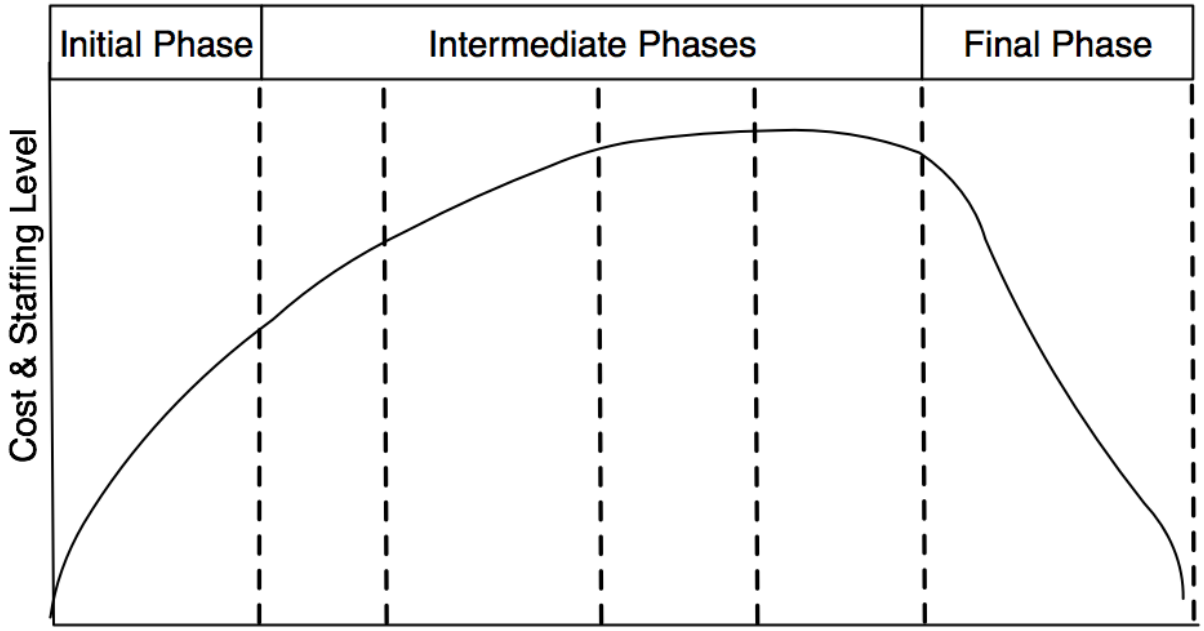
- In a *predictive life cycle*:
  - Product and deliverables are defined at the beginning of the project
  - Changes to scope are carefully–and restrictively–managed
- In an *iterative and incremental life cycle*:
  - Project phases repeat one or more project activities, taking advantage of increased understanding of the product
  - Each phase (and each iteration within a phase) successively adds to the functionality of the product
  - Scope is usually well-defined early in the project life cycle, but can be changed with relatively low overhead as project proceeds
- In an *adaptive life cycle [Agile]*:
  - Product is developed over multiple phases, each with several iterations
  - Detailed scope is defined for each phase only as the phase begins

IT project life cycles

- IT projects have two concurrent life cycles:
  - *Project life* cycle (PLC) encompasses all activities of project, including the System/Software Development Life Cycle (SDLC)
  - PLC is directed toward achieving *project* requirements
  - SDLC is directed toward achieving *product* requirements
- Both life cycle models are needed to manage an IT project
  - PLC alone will not adequately address system development concerns
  - SDLC alone will not adequately address business and product integration concerns
  - Effective integration of the two life cycle models is essential to improving the likelihood of project success
- In effect, the PLC and the SDLC should be so closely interwoven that they need not be distinguished from each other

What is a project life cycle?

- Consists of a number of generally sequential phases
- Phases are defined by technical information transfer or technical component hand-off
- Cost and staffing levels vary as a function of time according to the following qualitative schematic diagram:



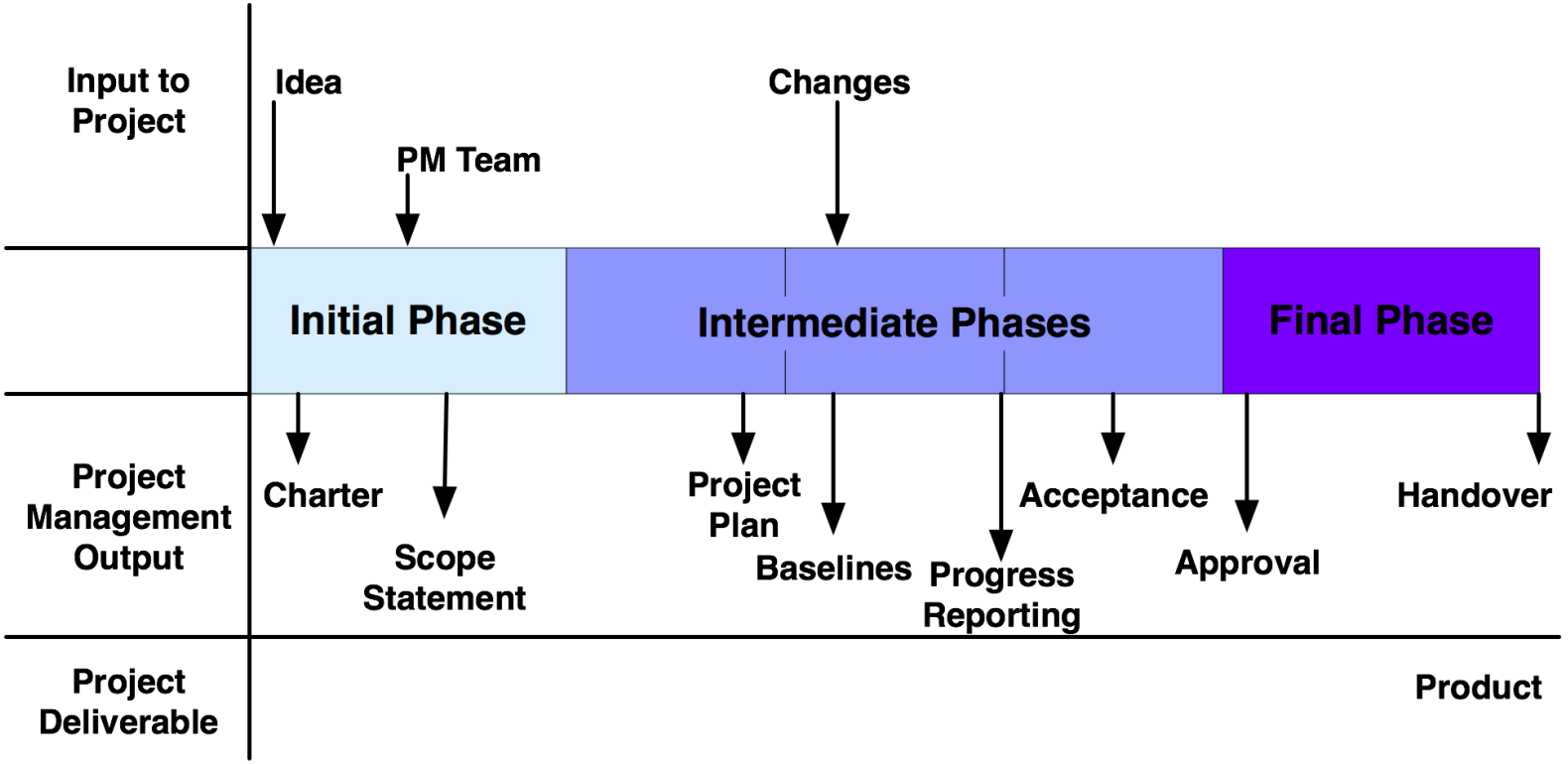
Adapted from PMBOK Guide 3rd Edition Fig. 2-1

What is a project life cycle?

- Risk of failure is greatest at start of project when the level of uncertainty is highest
- Stakeholder influence over project /product decreases as project continues
- Project life cycles define:
  - Technical work to be done in each phase
  - When deliverables are to be generated in each phase
  - How each deliverable is reviewed, verified, and validated
  - Who is involved in each phase
  - How to control each phase
  - How to approve each phase

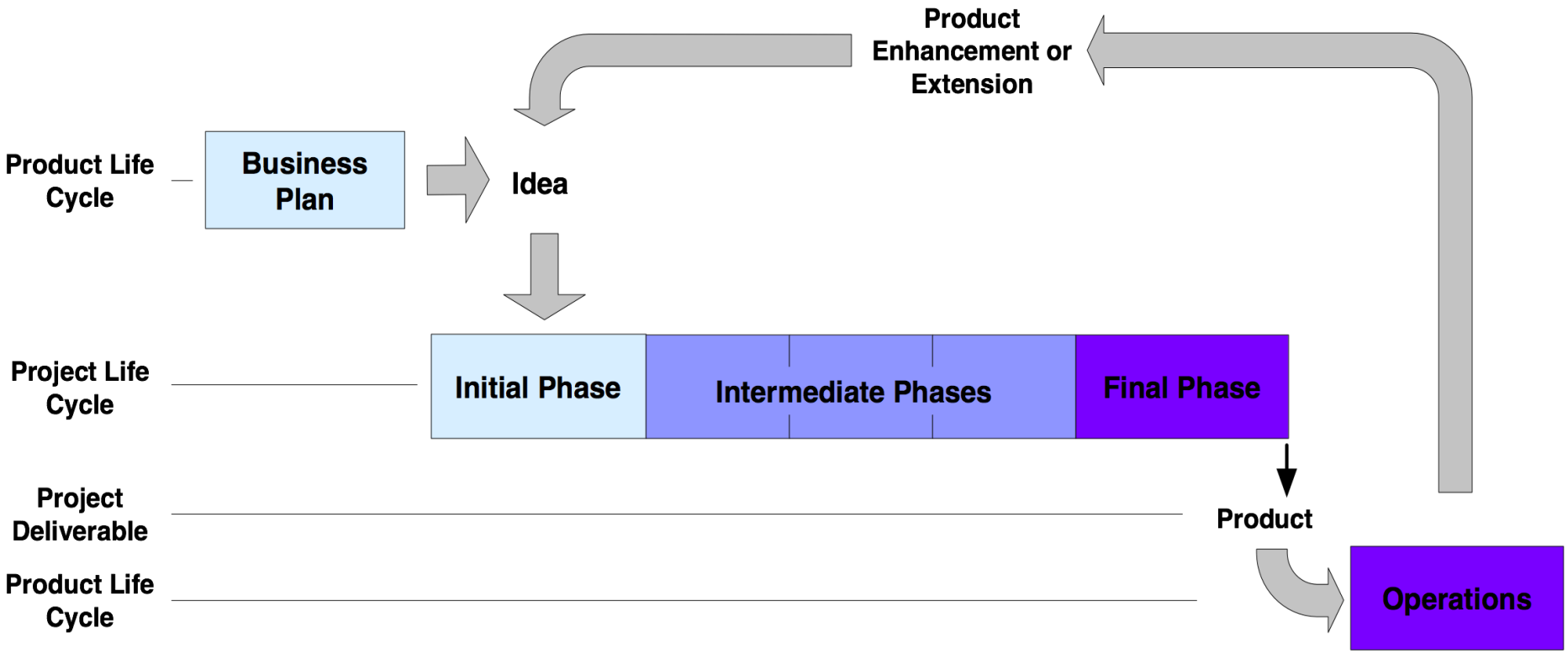
Phases in project life cycle

- The completion and approval of one or more deliverables (measurable, verifiable work product) defines a project phase
- In **iterative systems development**, new phase can be started without closing the previous phase
- A phase can be closed without initiating subsequent phase



Adapted from PMBOK Guide 3rd Edition Fig. 2-3

Project & product life cycles



Adapted from PMBOK Guide 3rd Edition Fig. 2-4

The systems development lifecycle

- "The *systems development life cycle (SDLC) is the process of* understanding how an information system (IS) can support business needs by designing a system, building it, and delivering it to users"*
- A *methodology is a formalized approach to implementing the SDLC*
- What differentiates one methodology from another:
  - The specific activities that must be performed
  - When, how, and how often the activities are performed
  - Who performs the activities
  - The amount of emphasis placed on an activity at a specific point in time

* Dennis, Alan (2012-05-01). *Systems Analysis and Design with UML, 4th Edition (Page 2). Wiley. Kindle Edition.*

Software Development Process

- Prescriptive
  - Linear/sequential (Classic and Waterfall)
  - Evolutionary (Iterative/incremental or spiral)
  - Unified Process (Unified Process is **based on the enlargement and refinement of a system through multiple iterations**, with cyclic feedback and adaptation.)
- Adaptive
  - Lean and agile methods

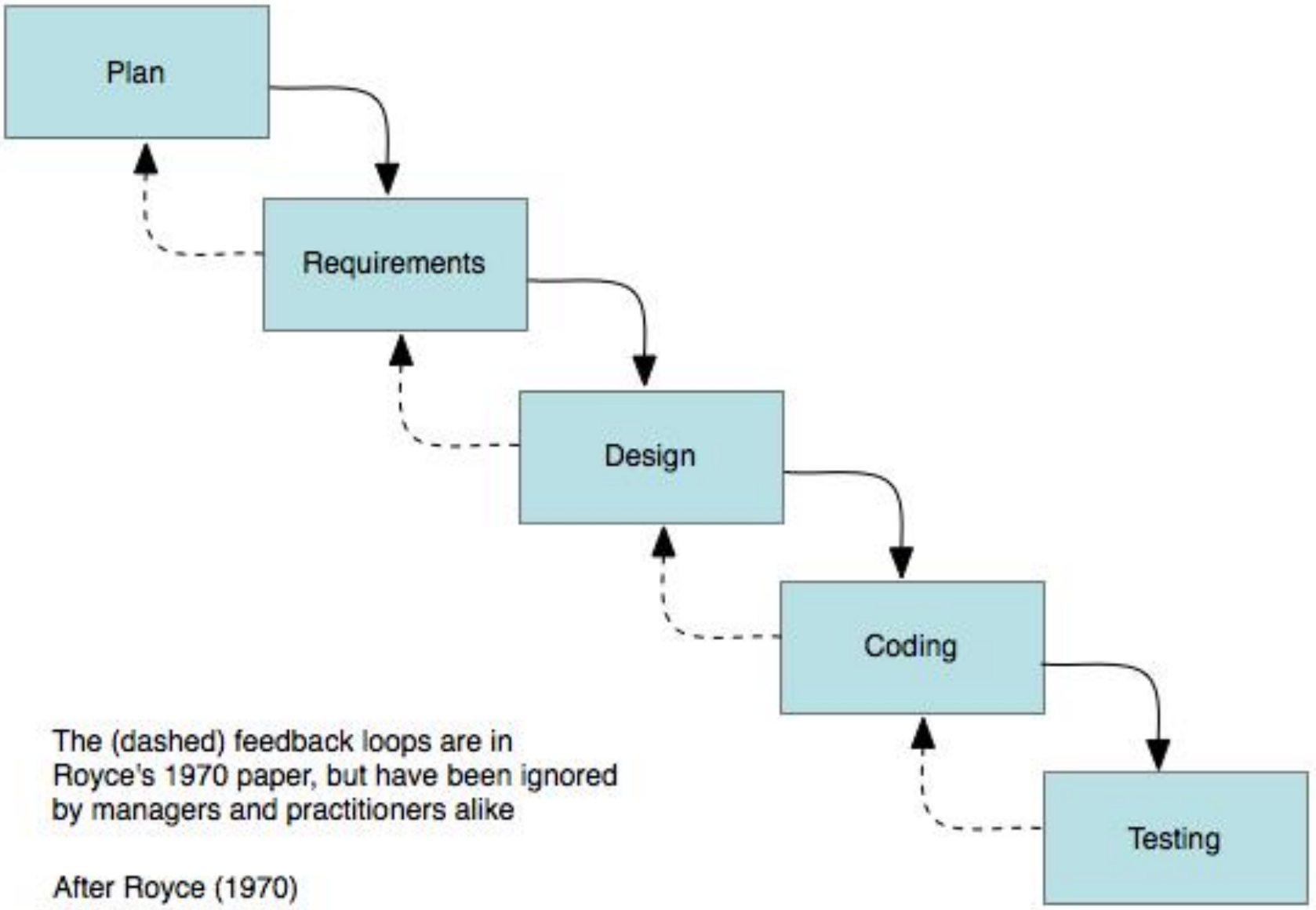
Sequential (‘waterfall’) methodology

- The term *waterfall* was coined by Winston Royce in a 1970 paper titled *Managing the Development of Large Software Systems*, in the Proceedings of IEEE WESCON
- The paper used the sequential waterfall approach as an example of an ***ill-conceived, risk-prone practice*** for developing large systems
- Royce advocated a series of iterative feedback loops among the development stages, incrementally gaining learning value from working software
- Instead of adopting the approach Royce advocated, managers and practitioners adopted its *anti-form, without* feedback loops

Waterfall SDLC

- Each phase is marked by completion of Deliverables
- The primary software project phases:
  - Requirements
  - Analysis
  - Design
  - Construction
  - Quality Assurance (aka Testing)
  - Deployment

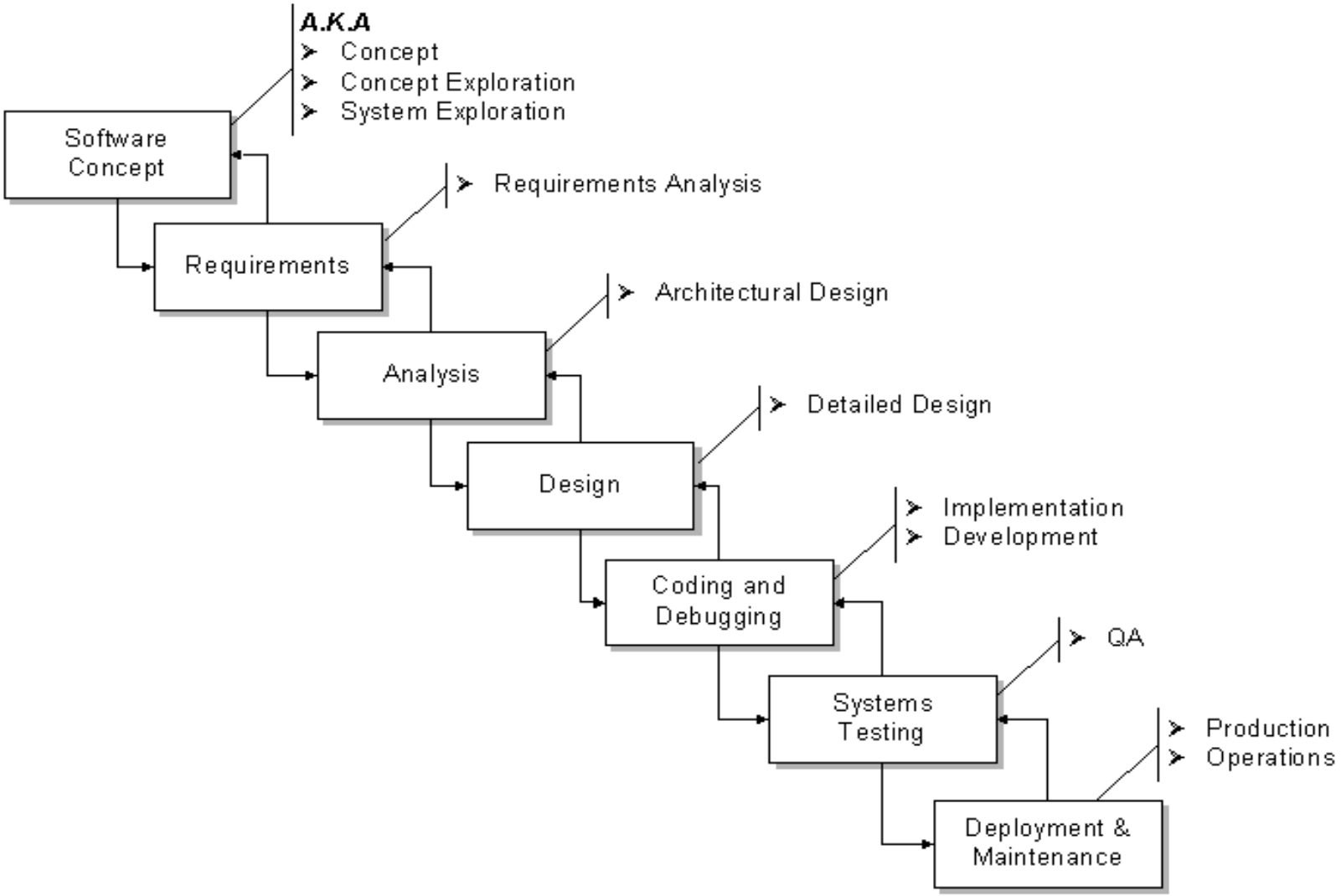
Waterfall SDLC



The (dashed) feedback loops are in Royce's 1970 paper, but have been ignored by managers and practitioners alike

After Royce (1970)

Project Phases A.K.A.



Waterfall system development model

- Highly-sequential process
- Failure symptoms:
  - Extended integration and late design breakage
  - Late risk resolution
  - Requirements-driven functional decomposition
  - Argumentative stakeholder relationships
  - Focus on documents and review meetings
- Still followed (in name or practice) by many organizations, usually a modified version

Waterfall system development model

Sequential: suitable projects and management approaches

- A sequential SDLC is suitable for projects with:
  - Clear, unambiguous, and stable user requirements
  - Familiar, proven technology
  - Low complexity
  - Adequate time
  - Stable schedule
- A project meeting most of these criteria can use conventional project management practices, such a big, up-front planning and conventional risk assessment

Evolutionary methodologies

- An *evolutionary methodology* follows an iterative and incremental approach that allows the start of development with incomplete, imperfect knowledge
- An iterative and incremental process is like solving a jigsaw puzzle: neither top-down nor bottom-up but **buildup** and **converge**
- An iterative and incremental process offers these advantages:
  - Logical progress toward evolving a robust architecture
  - Effective management of changing requirements
  - Effective means to address changes in planning
  - Ability to perform continuous integration
  - Early understanding of the system (the ‘Hello world!’ effect)
  - Ongoing risk assessment
- Evolutionary methodologies are incremental at both the macro (project-scale) and micro (working team) process levels

Iterative system development model

- Non-linear approach to system development
- Incorporates top five principles of modern development processes:
  - *Architecture first*. Provides the central design element
  - *Iterative life-cycle process*. Provides the essential risk management element
  - *Component-based development*. Provides the technology element
  - *Change management environment.* Provides the control element
  - *Round-trip engineering*. Provides the automation element

5,000 foot view of Iterative SDLC

- **Iterative** SD model defines four life-cycle phases:
  - Inception
  - Elaboration
  - Construction
  - Transition
- We **iterate** through each phase, and repeat as needed.
- Now, for a quick survey of the phases…

Inception phase

- Essential activities
  - *Formulate product scope*. Capture requirements and operational concept
  - *Perform feasibility analysis*. Determine whether the organization has the resources and technical capabilities to meet customer's needs
  - *Synthesize the system architecture*. Evaluate essential system design constraints and trade-offs, as well as available solutions
  - *Plan and prepare business case*. Address risk management, staffing, iteration plans, cost, and infrastructure

Elaboration phase

- Most critical phase of the four
- Essential activities
  - *Elaborate the vision*. Detail elements of the vision that drive architectural or planning decisions
  - *Elaborate the process and infrastructure*. The construction process and environment are established here
  - *Elaborate the architecture and select reusable (internal or COTS) components*. Baseline the architecture as quickly as possible and demonstrate that the architecture will support the vision at reasonable cost in reasonable time

Construction phase

- Essential activities
  - *Achieve useful versions* (intermediate, alpha, beta, and other test releases)
  - *Perform resource management, control, and process optimization*
  - *Complete component development and test*
  - *Assess product releases against acceptance criteria*

Transition phase

- Essential activities
  - *Perform deployment-specific engineering tasks*. Commercial packaging and production, sales kit development, field personnel training
  - *Assess deployment baselines against complete vision and acceptance criteria.* Examine and compare what is being delivered to what was envisioned and delineated by acceptance criteria
  - Plan for next iteration

Comparative expenditure profiles

Waterfall		Iterative	
Activity	**Cost**	**Cost**	**Activity**
Management	5%	10%	Management
Requirements	5%	10%	Requirements
Design	10%	15%	Design
Code & Unit Testing	30%	25%	Implementation
Integration & Test	40%	25%	Assessment
Deployment	5%	5%	Deployment
Environment	5%	10%	Environment
Total	100%	100%	Total

Based on and adapted from Tables 1-1 and 10-1 in
Software Project Management: A Unified Approach by Walker Royce

Suitable Projects And Management Approaches

- An evolutionary SDLC is suitable for projects with:
  - Reasonably–but not perfectly–clear user requirements
  - Unfamiliar or unproven technology
  - High complexity
  - Short time schedule
  - Schedule variability
- Such a project would use rolling wave planning rather than big, up-front planning and use a continuous, adaptive approach to risk assessment and management

Agile Project Management

Agile Projects

- Lean methodology. Only as much process as necessary.
- 'Agile' is an umbrella term used for identifying various models used for agile development, such as Scrum.
- Scrum is **an agile project management framework** that helps teams structure and manage their work through a set of values, principles, and practices
- Since agile development model is different from conventional models, agile project management is a specialized area in project management.

Agile Projects

- Agile project management is an iterative approach to planning and guiding project processes.
- An agile project is completed in small sections called iterations, or, in scrum, sprints.
- Each iteration is reviewed and critiqued by the project team, which may include representatives of the client business as well as employees.
- Insights gained from the critique of an iteration are used to determine what the next step should be in the project.
- Each project iteration is typically scheduled to be completed within two weeks.

Agile Project Steps

1. The **product owner** identifies the **product vision**.
2. The **product owner** creates a **product roadmap**.
3. The **product owner** creates a **release plan**.
4. The **product owner**, the (scrum) **master**, and the **development team** plan sprints, also called iterations, and start creating the product within those sprints
5. During each sprint, the development team has daily meetings [called scrums].
6. The team holds a sprint review.
7. The team holds a sprint retrospective.

Agile Project Artifacts

1. **Product vision statement:** An elevator pitch, or a quick summary, to communicate how your product supports the company's or organization's strategies. The vision statement must articulate the goals for the product. Revisit once a year.
2. **Product roadmap:** The product roadmap is a high-level view of the product requirements, with a loose time frame for when you will develop those requirements. Revisit twice a year.
3. **Release plan:** A high-level timetable for the release of working software.
4. **Product backlog:** The full list of what is in the scope for your project, ordered by priority. Once you have your first requirement, you have a product backlog.
5. **Sprint backlog:** The goal, user stories, and tasks associated with the current sprint.
6. **Increment:** The working product functionality at the end of each sprint.

Agile Project Roles

1. **Development team:** The group of people who do the work of creating a product. Programmers, testers, designers, writers, and anyone else who has a hands-on role in product development is a member of the development team.
2. **Product owner:** The person responsible for bridging the gap between the customer, business stakeholders, and the development team. The product owner is sometimes called a *customer representative.*
3. **Scrum master:** The person responsible for supporting the development team, clearing organizational roadblocks, and keeping the agile process consistent. A scrum master is sometimes called a project facilitator.
4. **Stakeholders:** Anyone with an interest in the project.
5. **Agile mentor:** Someone who has experience implementing agile projects and can share that experience with a project team. The agile mentor can provide valuable feedback and advice to new project teams and to project teams that want to perform at a higher level.

Agile Project Events

1. **Project planning:** The initial planning for your project.
   - includes creating a **product vision** statement and a **product roadmap,**
   - can take place in as little time as one day.
2. **Release planning:** Planning the next set of product features to release
3. **Sprint:** A short cycle of development, in which the team creates potentially shippable product functionality.
4. **Sprint planning:** A meeting at the beginning of each sprint where the scrum team commits to a sprint goal.
5. **Daily scrum:** A 15-minute meeting held each day in a sprint, where development team members state what they completed the day before, what they will complete on the current day, and whether they have any roadblocks.

Agile Project Events

6. **Sprint review:** A meeting at the end of each sprint, where the development team demonstrates the working product functionality it completed during the sprint.
7. **Sprint retrospective:** A meeting at the end of each sprint where the scrum team discusses what went well, what could change, and how to make any changes.

Selection considerations: guiding questions

- Organizational characteristics
  - What are the characteristics of the organizational culture? What are the management comfort levels with the various methodologies?
  - How open is management and the organization to change?
  - Is the organization risk-tolerant or risk-adverse?
  - What is the organization's tolerance for real risk vs. perceived risk?
- Project characteristics
  - How large is the project?
  - What is the project's estimated duration?
  - Are teams co-located or distributed?
  - Is regulatory compliance a significant factor?
  - How flexible are documentation requirements?

Selection considerations: guiding questions

- People and management characteristics
  - What are the experience levels of team members?
  - Are team members self-motivated or command-driven?
  - What sort of management style is employed? Laissez-faire, micromanagement, or somewhere in-between?
  - What sort of social dynamics govern project efforts within the organization? Cooperative and problem-solving, adversarial, or blaming?

Methodology characteristics compared

System Characteristics	'Waterfall'	Plan-Driven Evolutionary	Agile Evolutionary
Unclear user requirements	Poor	Good	Excellent
Unfamiliar technology	Poor	Good	Good
Complex	Fair	Very Good	Good
Reliable	Good	Good	Excellent
Short time schedule	Poor	Excellent	Excellent
Schedule variability	Poor	Excellent	Excellent

Examples: Applying the table

1. Short time schedule + shifting user requirements
   - Agile
2. Complex + short time schedule
   - Iterative
3. Clear user requirements + long time schedule + command-driven team
   - Water-fall
4. Reliable + complex + schedule variability
   - Agile
5. Unfamiliar technology + short time schedule + schedule variability
   - Either Agile or Iterative